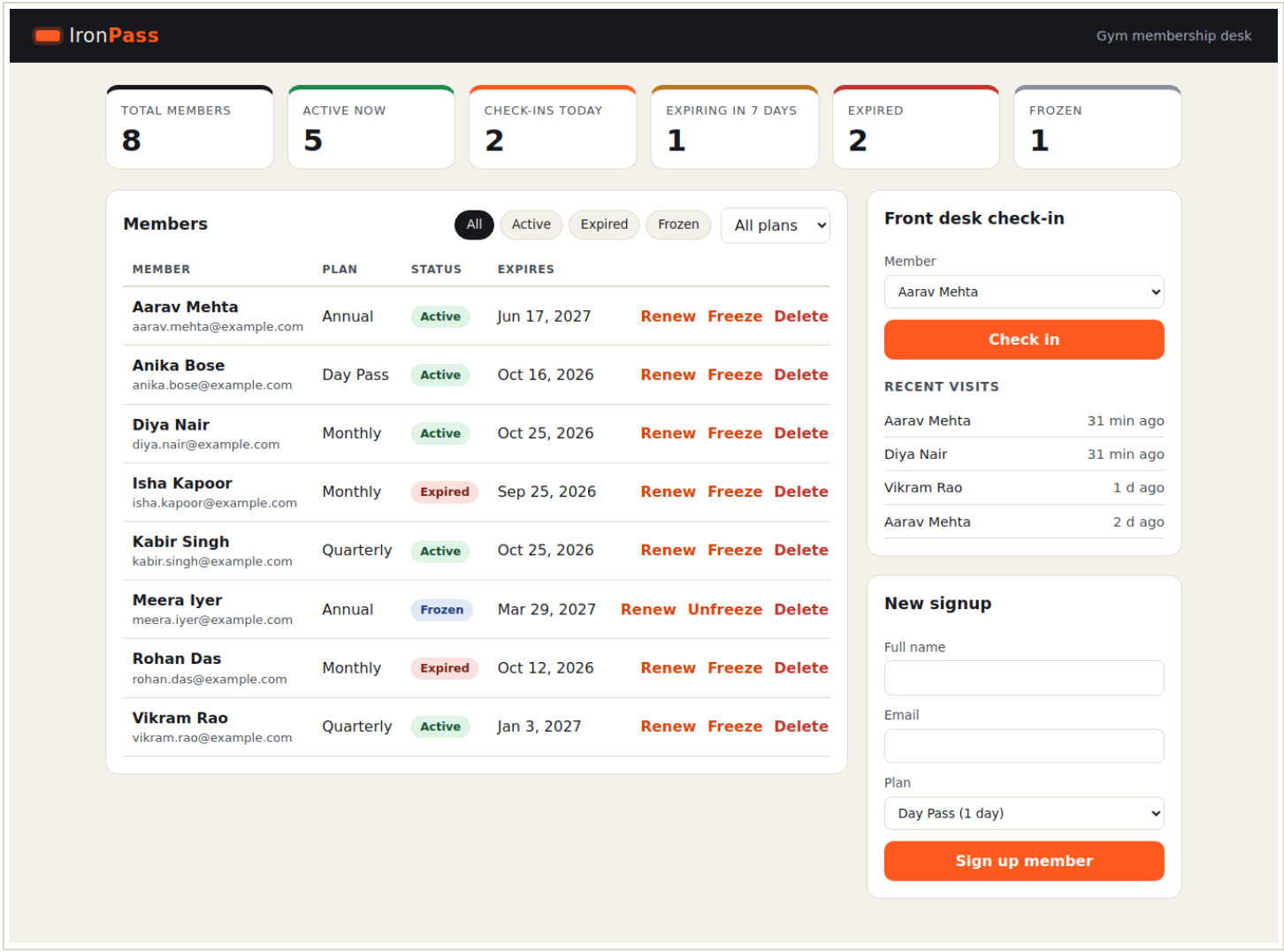
IronPass

From a gym membership app to a monitored Kubernetes cluster

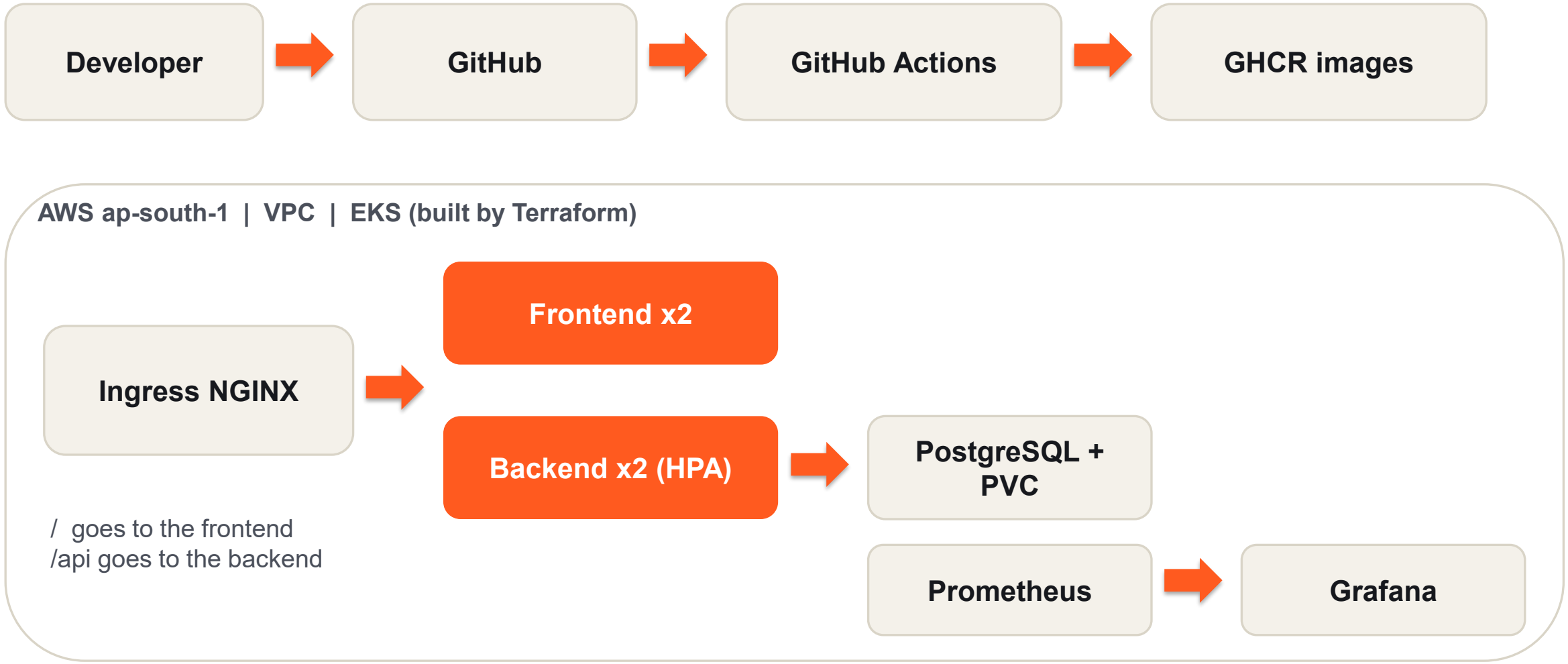
DevOps final capstone | Romit

The problem and the app

- Members, plans and check-ins
- Rule: expired plan means no check-in
- React + FastAPI + PostgreSQL
- 12 REST endpoints, /health, /ready, /metrics



Architecture



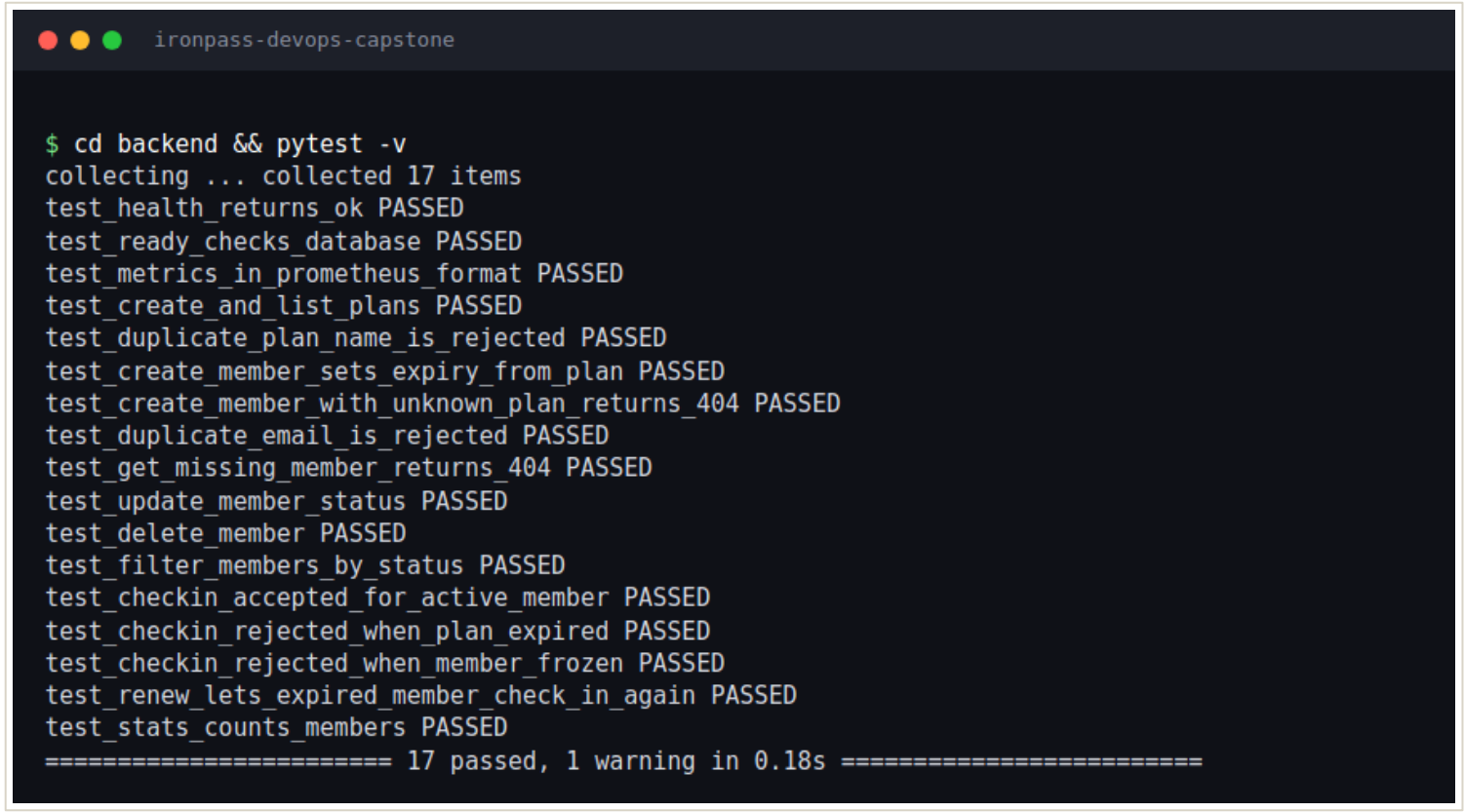
Run it locally with Docker

- docker compose up --build
- 3 services: frontend, backend, postgres
- Frontend: multi-stage, unprivileged nginx
- Both images run as non-root



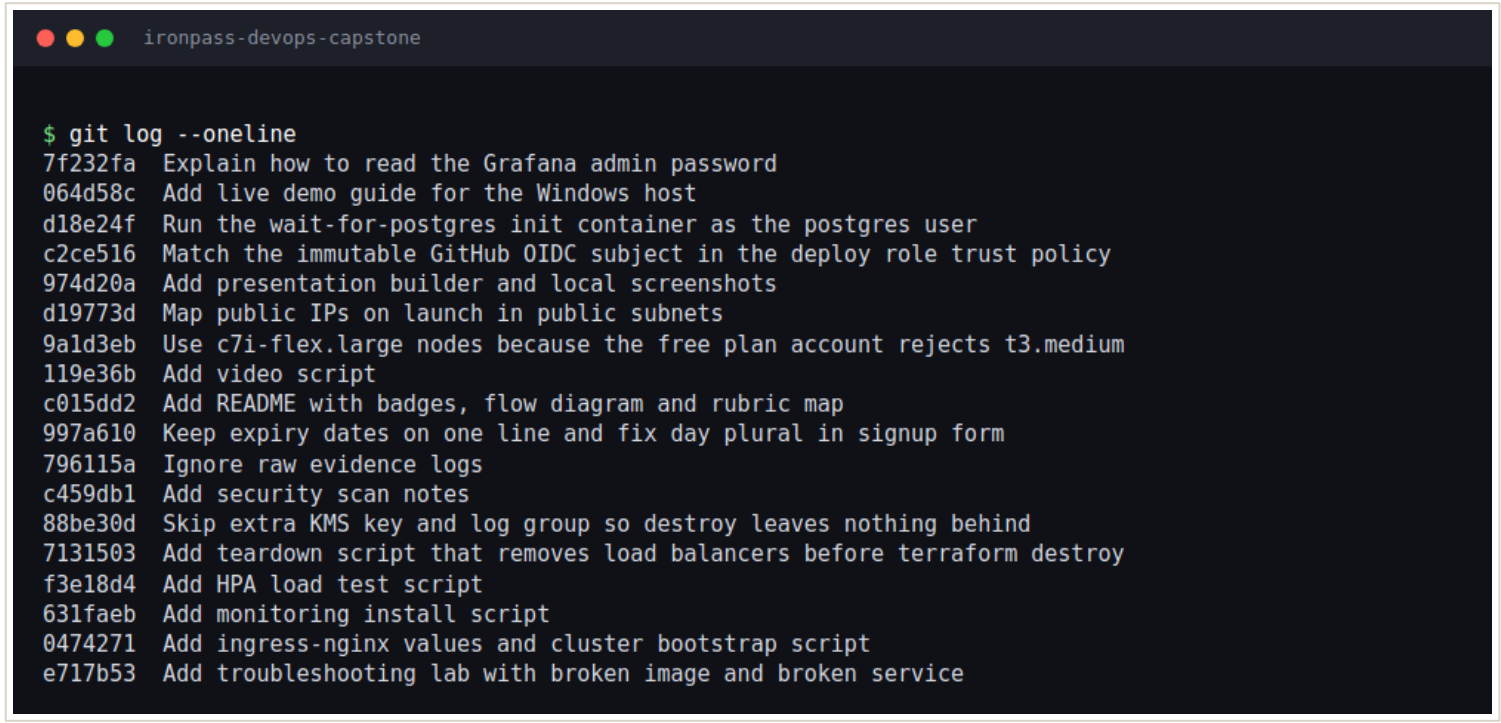
Tests

- 17 tests, all passing
- SQLite test database
- Rule tested: expired plan is rejected
- Runs first in the pipeline



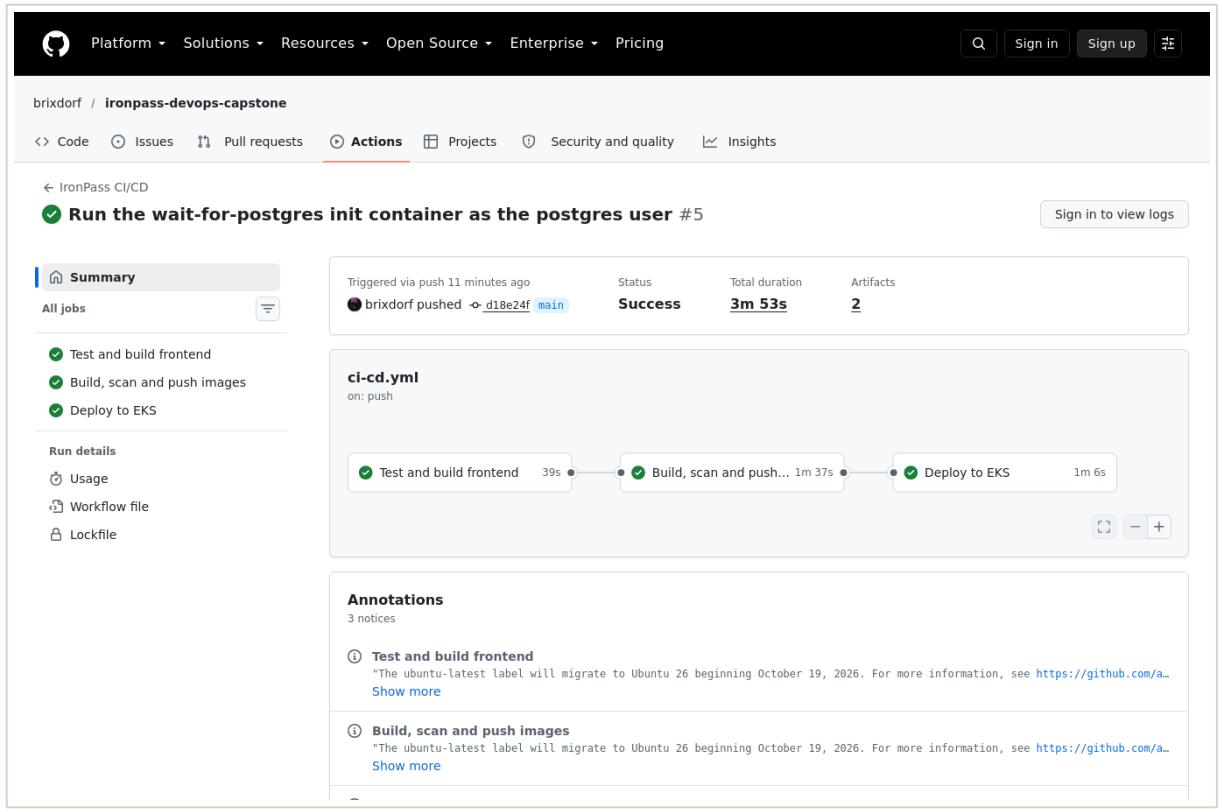
Git and GitHub

- Small commits, one change each
- Plain messages
- .gitignore: .env, state, node_modules
- No secrets in history



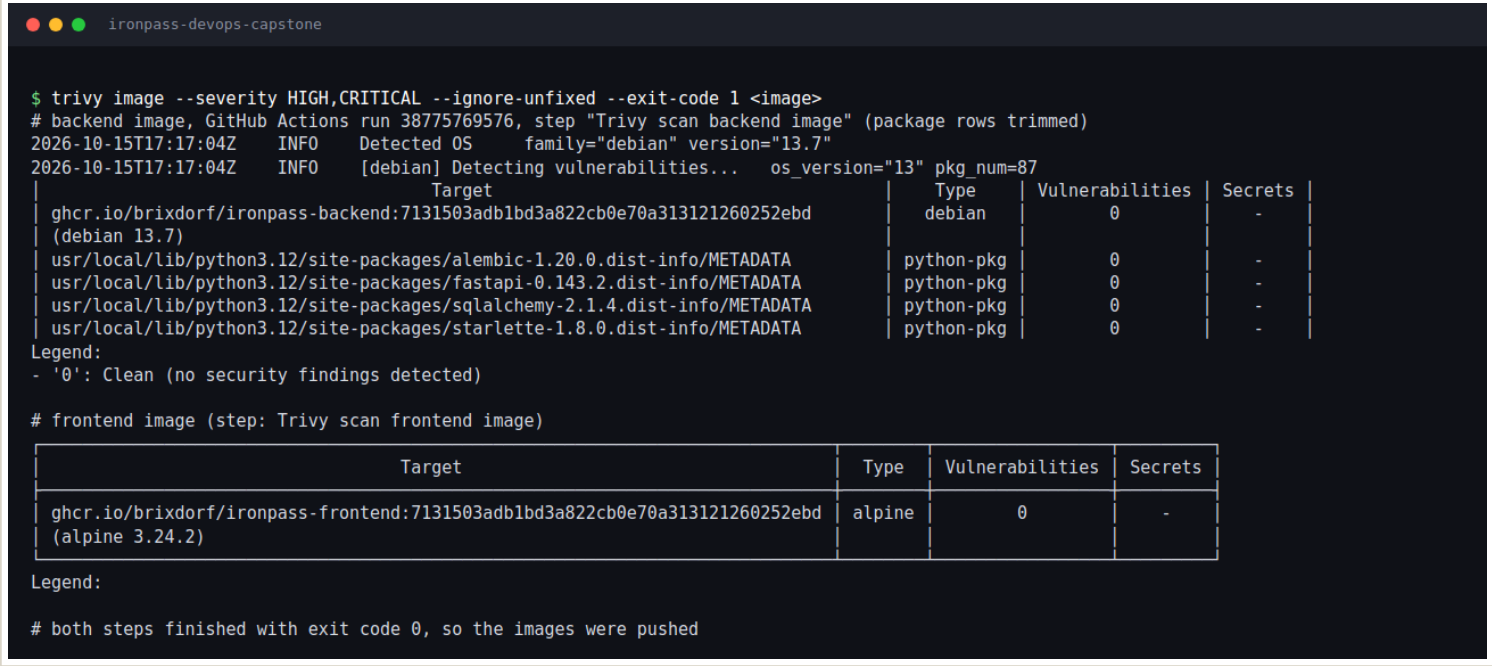
CI/CD pipeline

pytest → Build frontend → Build images → **Trivy scan** → Push to GHCR → Helm deploy



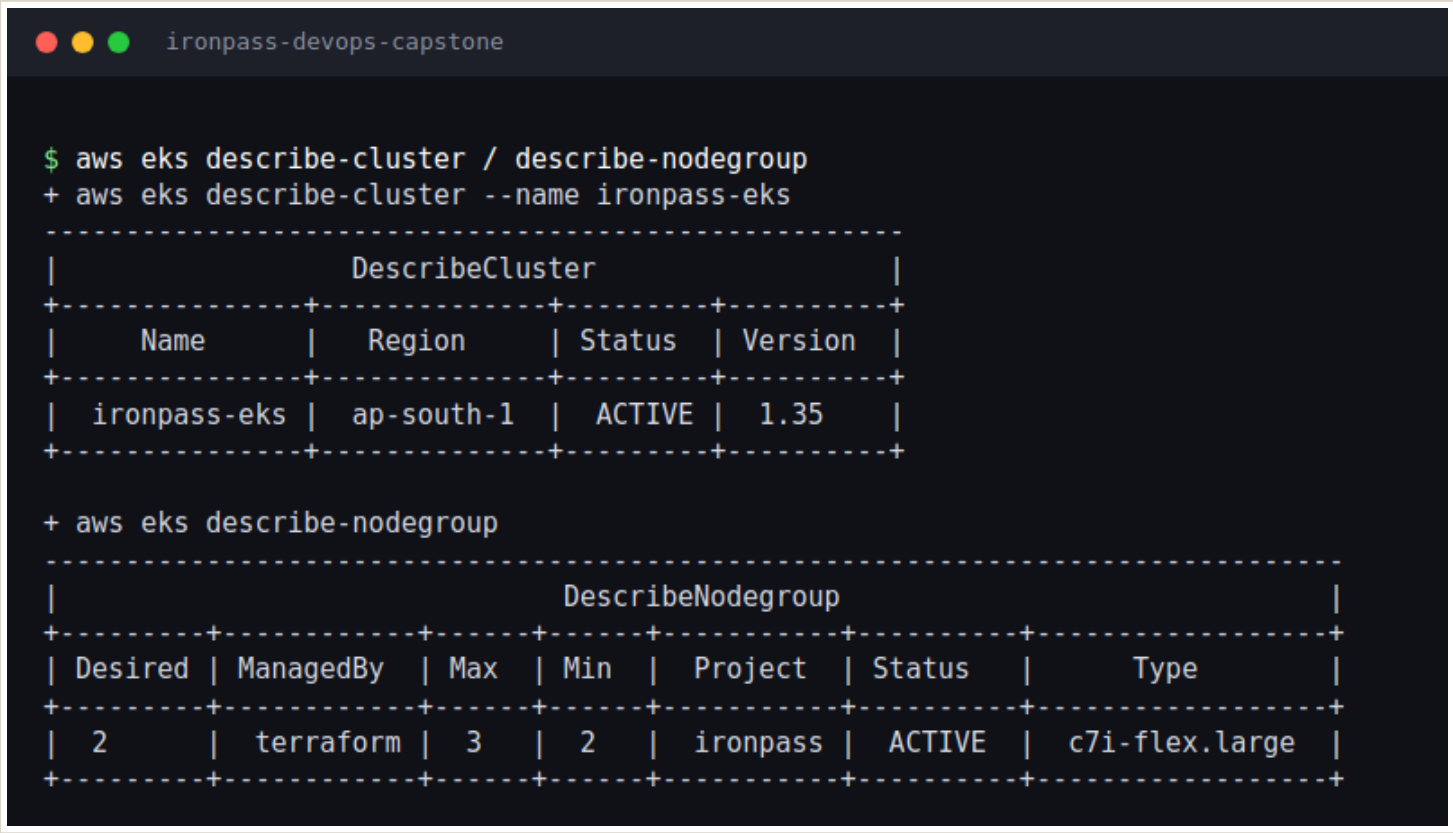
Security scan with Trivy

- HIGH and CRITICAL, fixable only
- Fails the build
- First runs failed: 7 and 41 findings
- Fixed, now clean



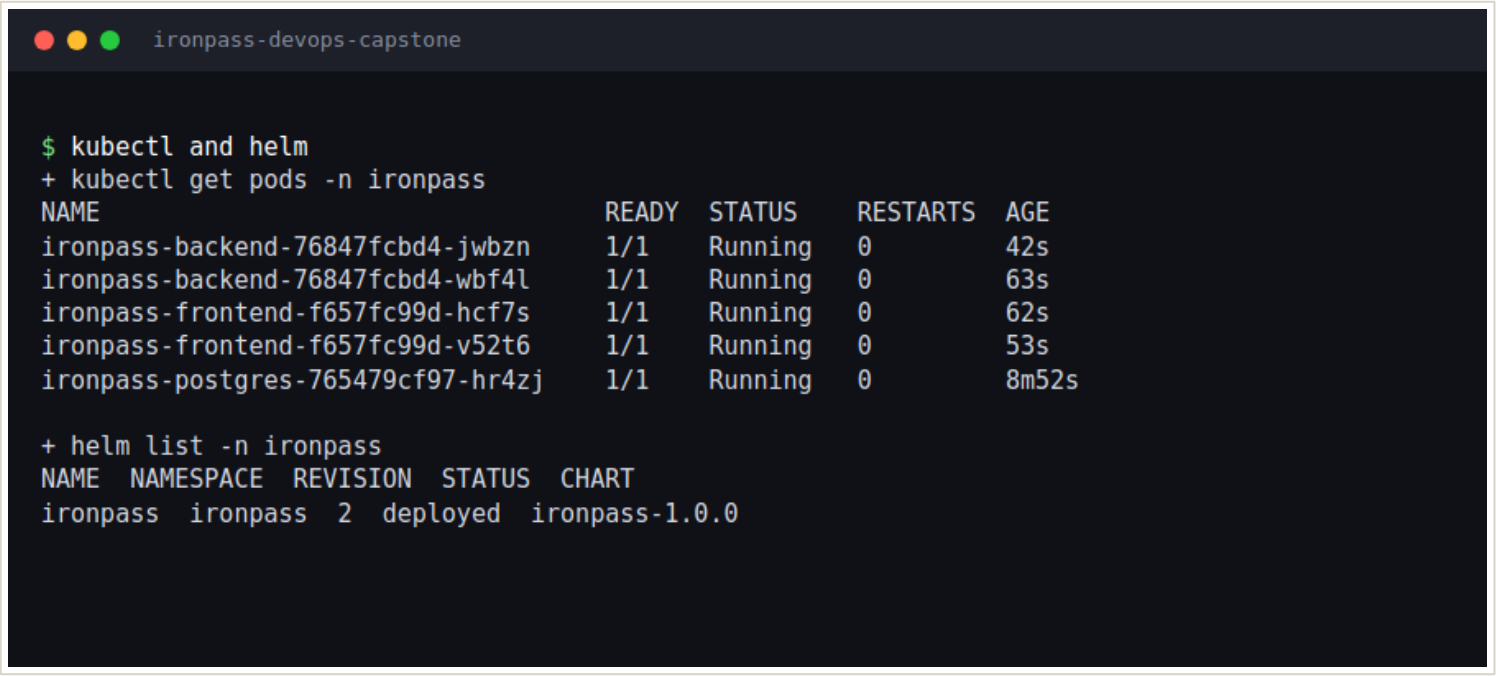
Terraform and AWS

- VPC: 2 public + 2 private subnets
- One NAT gateway
- EKS with managed nodes, ap-south-1
- GitHub OIDC role, no AWS keys

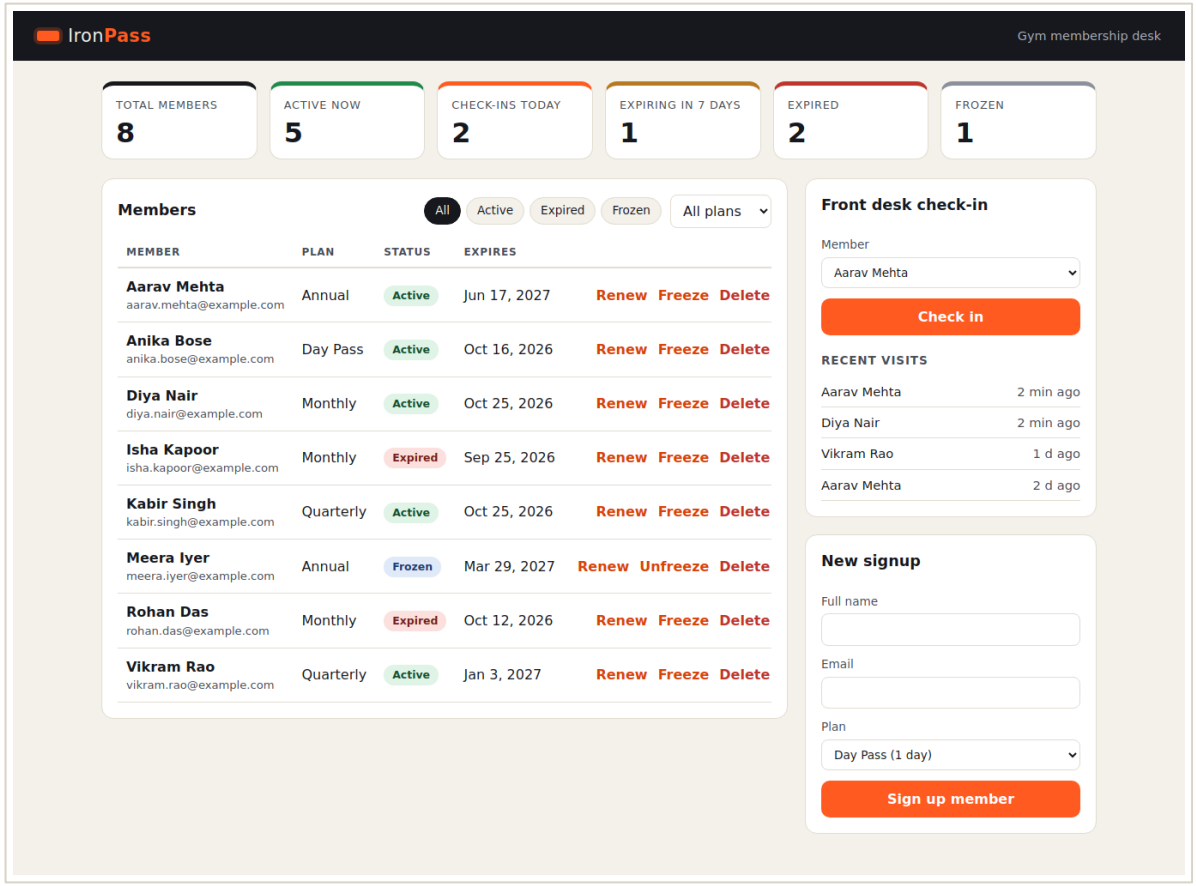


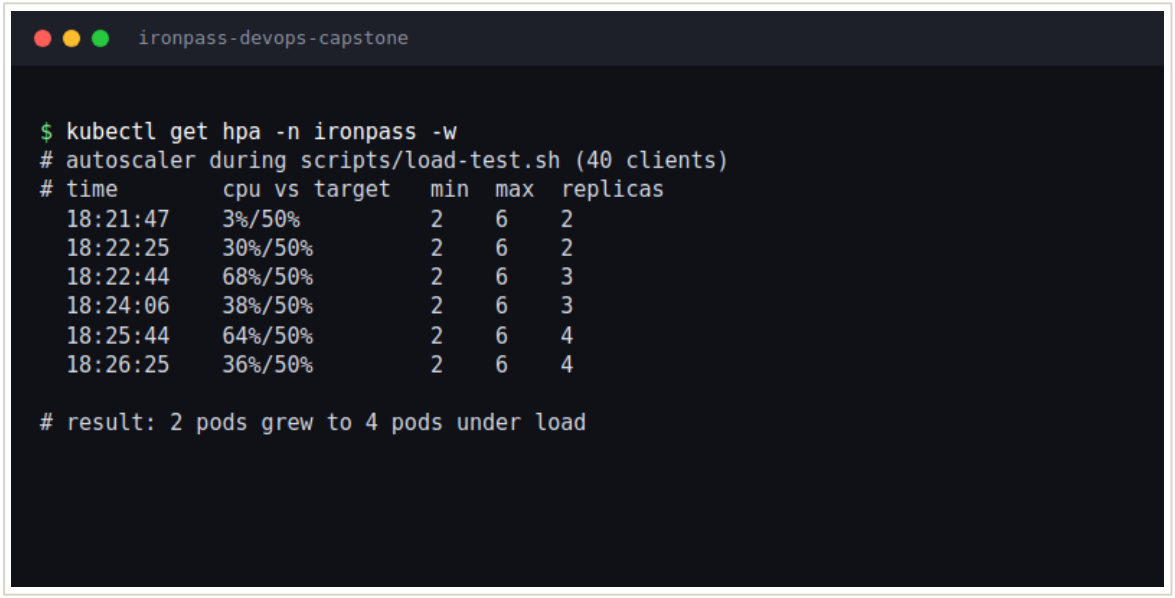
Kubernetes and Helm

- One Helm chart, prod and dev values
- 2 replicas each, ClusterIP services
- Probes: /health and /ready
- Postgres with a PVC

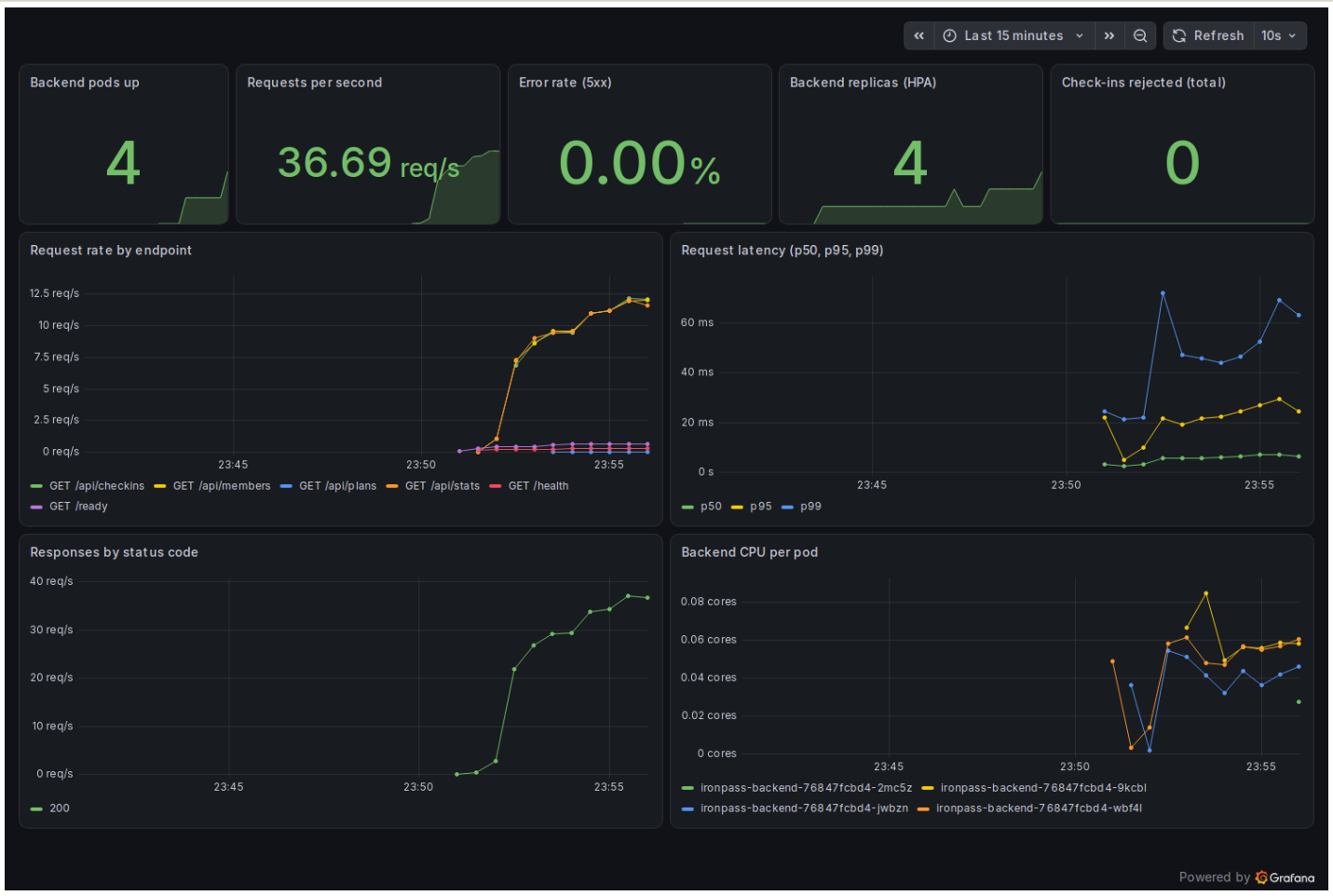


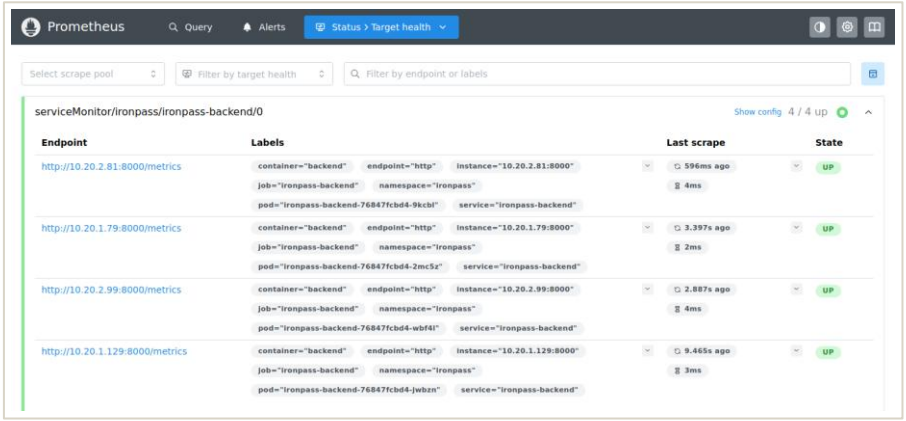
Ingress and autoscaling





Monitoring

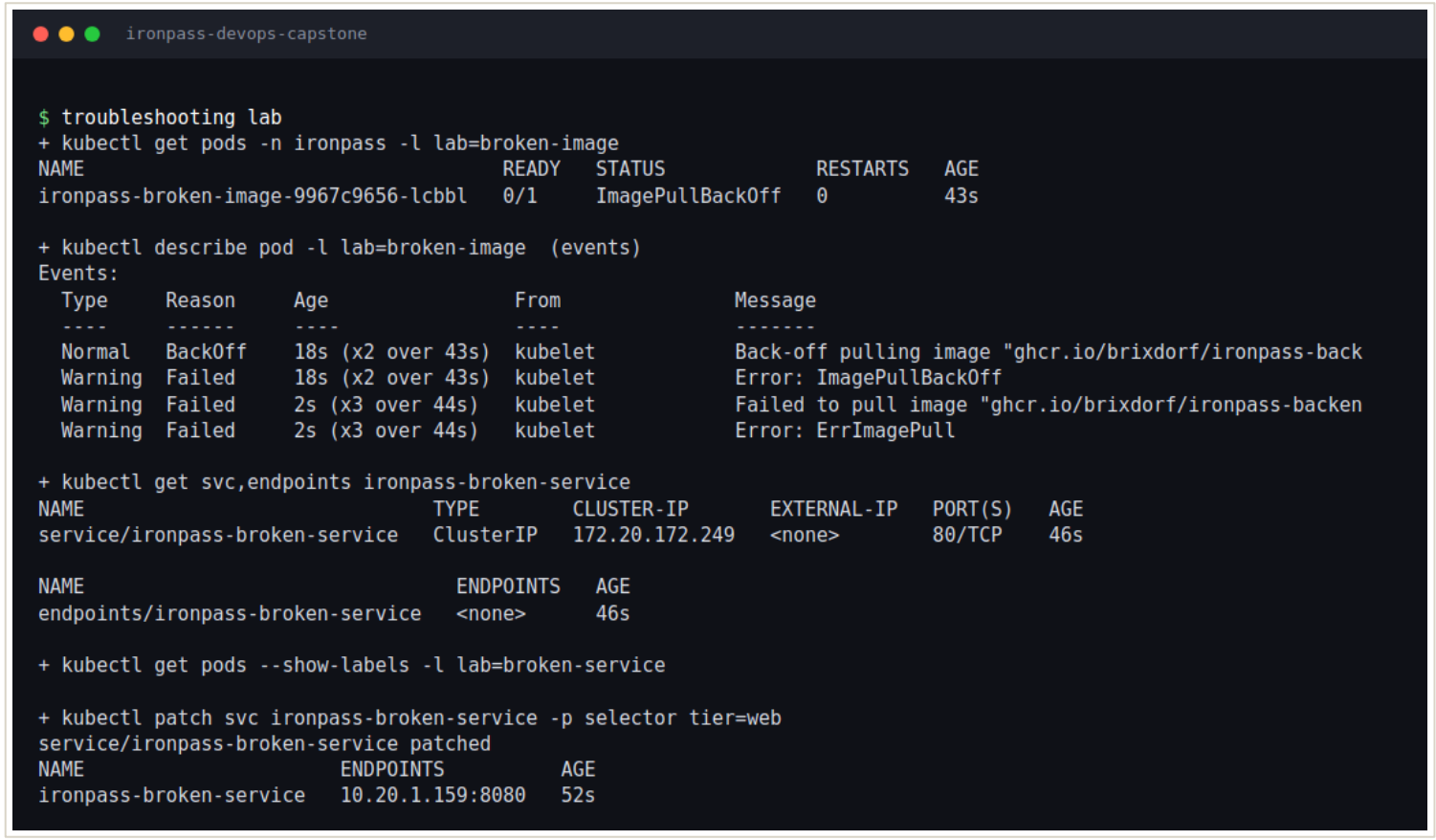




Target ironpass-backend
4 of 4 UP
Dashboard is provisioned from a ConfigMap

Troubleshooting lab

- Broken image: ImagePullBackOff
- Broken service: no endpoints
- Tools: describe, events, endpoints
- Fix notes in troubleshooting/



What I learned

- Every tool has one job, the pipeline joins them
- A failing scan is useful, it made me fix real issues
- Infrastructure as code makes the cluster repeatable
- Probes, requests and metrics make an app operable
- Cost matters: tag everything and tear down in order